```
import numpy as np

house_data = np.array([
    [3, 1400, 7500000],
    [5, 2200, 12500000],
    [4, 1800, 9800000],
    [6, 3000, 18500000],
    [2, 1100, 5200000]
])

print("Original house_data array:")
print(house_data)

bedrooms_column_index = 0
sale_price_column_index = 2

houses_more_than_four_bedrooms = house_data[house_data[:, bedrooms_column_index] > 4]

print("\nHouses with more than four bedrooms:")
print(houses_more_than_four_bedrooms)

if houses_more_than_four_bedrooms.shape[0] > 0:
    average_sale_price = np.mean(houses_more_than_four_bedrooms[:, sale_price_column_index])
    print(f"\nAverage sale price of houses with more than four bedrooms: {average_sale_price:,.2f}")
else:
    print("\nNo houses found with more than four bedrooms to calculate the average sale price.")
```

```
Original house_data array:
[[       3     1400  7500000]
 [       5     2200 12500000]
 [       4     1800  9800000]
 [       6     3000 18500000]
 [       2     1100  5200000]]

Houses with more than four bedrooms:
[[       5     2200 12500000]
 [       6     3000 18500000]]

Average sale price of houses with more than four bedrooms: 15,500,000.00
```

```
import numpy as np
from sklearn.linear_model import LogisticRegression

data = np.array([
    [1, 3, 120, 0],
    [6, 15, 300, 1],
    [0, 1, 80, 0],
    [4, 10, 250, 1],
    [2, 5, 160, 0]
])

X = data[:, :3]
y = data[:, 3]

print("Features (X):\n", X)
print("\nTarget (y):\n", y)

model = LogisticRegression()
model.fit(X, y)

print("\nLogistic Regression model trained successfully.")

new_email_features = np.array([[3, 8, 200]])

prediction = model.predict(new_email_features)
prediction_proba = model.predict_proba(new_email_features)

print(f"\nNew Email Features: {new_email_features[0]}")
print(f"Predicted class (0=Not Spam, 1=Spam): {prediction[0]}")
print(f"Prediction probabilities (Not Spam, Spam): {prediction_proba[0]}")

if prediction[0] == 1:
    print("The new email is predicted to be SPAM.")
else:
    print("The new email is predicted to be NOT SPAM.")
```

```
Features (X):
 [[  1   3 120]
 [  6  15 300]
 [  0   1  80]
 [  4  10 250]
 [  2   5 160]]

Target (y):
 [0 1 0 1 0]

Logistic Regression model trained successfully.
```

```
New Email Features: [  3   8 200]
Predicted class (0=Not Spam, 1=Spam): 0
Prediction probabilities (Not Spam, Spam): [0.67050216 0.32949784]
The new email is predicted to be NOT SPAM.
```

```
import numpy as np
from sklearn.linear_model import LogisticRegression

data = np.array([
    [1, 3, 120, 0],
    [6, 15, 300, 1],
    [0, 1, 80, 0],
    [4, 10, 250, 1],
    [2, 5, 160, 0]
])

X = data[:, :3]
y = data[:, 3]

model = LogisticRegression()
model.fit(X, y)

new_email_features = np.array([[3, 8, 200]])

prediction = model.predict(new_email_features)
prediction_proba = model.predict_proba(new_email_features)

display(X)
display(y)
display(prediction)
display(prediction_proba)

if prediction[0] == 1:
    print("The new email is predicted to be SPAM.")
else:
    print("The new email is predicted to be NOT SPAM.")
```

```
array([[  1,   3, 120],
       [  6,  15, 300],
       [  0,   1,  80],
       [  4,  10, 250],
       [  2,   5, 160]])
array([0, 1, 0, 1, 0])
array([0])
array([[0.67050216, 0.32949784]])
The new email is predicted to be NOT SPAM.
```

```
import numpy as np
from sklearn.linear_model import LinearRegression

data = np.array([
    [2, 25000, 1200, 650000],
    [5, 60000, 1500, 420000],
    [3, 40000, 1300, 520000],
    [7, 90000, 1800, 300000],
    [1, 15000, 1000, 720000]
])

X = data[:, :3]
y = data[:, 3]

model = LinearRegression()
model.fit(X, y)

print("Linear Regression model trained successfully.")

new_car_age = float(input("Enter Car Age (years): "))
new_car_kms = float(input("Enter Kilometers Driven: "))
new_car_engine_cc = float(input("Enter Engine Capacity (cc): "))

new_car_features = np.array([[new_car_age, new_car_kms, new_car_engine_cc]])

predicted_price = model.predict(new_car_features)

print(f"Estimated Resale Price: {predicted_price[0]:,.2f}")
```

```
Linear Regression model trained successfully.
Enter Car Age (years): 4
Enter Kilometers Driven: 5874
Enter Engine Capacity (cc): 150
Estimated Resale Price: 571,895.58
```

```
import numpy as np
from sklearn.tree import DecisionTreeClassifier, export_text

data = np.array([
    [45000, 720, 0, 1],
    [20000, 650, 1, 0]
```

```
    [60000, 780, 0, 1],
    [28000, 620, 2, 0],
    [52000, 710, 1, 1]
])

X = data[:, :3]
y = data[:, 3]

model = DecisionTreeClassifier(random_state=42)
model.fit(X, y)

new_applicant_features = np.array([[35000, 680, 1]])

prediction = model.predict(new_applicant_features)

print(f"New Applicant Features: Income={new_applicant_features[0][0]}, Credit_Score={new_applicant_features[0][1]}, Existing_Loans={new_app
if prediction[0] == 1:
    print("Predicted Loan Approval: Approved")
else:
    print("Predicted Loan Approval: Rejected")

feature_names = ['Income', 'Credit_Score', 'Existing_Loans']
tree_rules = export_text(model, feature_names=feature_names)
print("\nDecision Tree Rules:")
print(tree_rules)
```

```
New Applicant Features: Income=35000, Credit_Score=680, Existing_Loans=1
Predicted Loan Approval: Rejected

Decision Tree Rules:
|--- Credit_Score <= 680.00
|   |--- class: 0
|--- Credit_Score >  680.00
|   |--- class: 1
```